Advanced Shader

Programmable Graphics Pipeline

Fixed Function Pipeline

3D Apps

API commands

3D API: OpenGL

GPU cmd & data stream

Vtx index

Assembled polygons

Pixel location

Pixel updates

GPU Frontend

Primitive Assembly

Rasterization & Interpolation

Raster Operations

Frame Buffer

Transformed vertices

Transformed Fragments

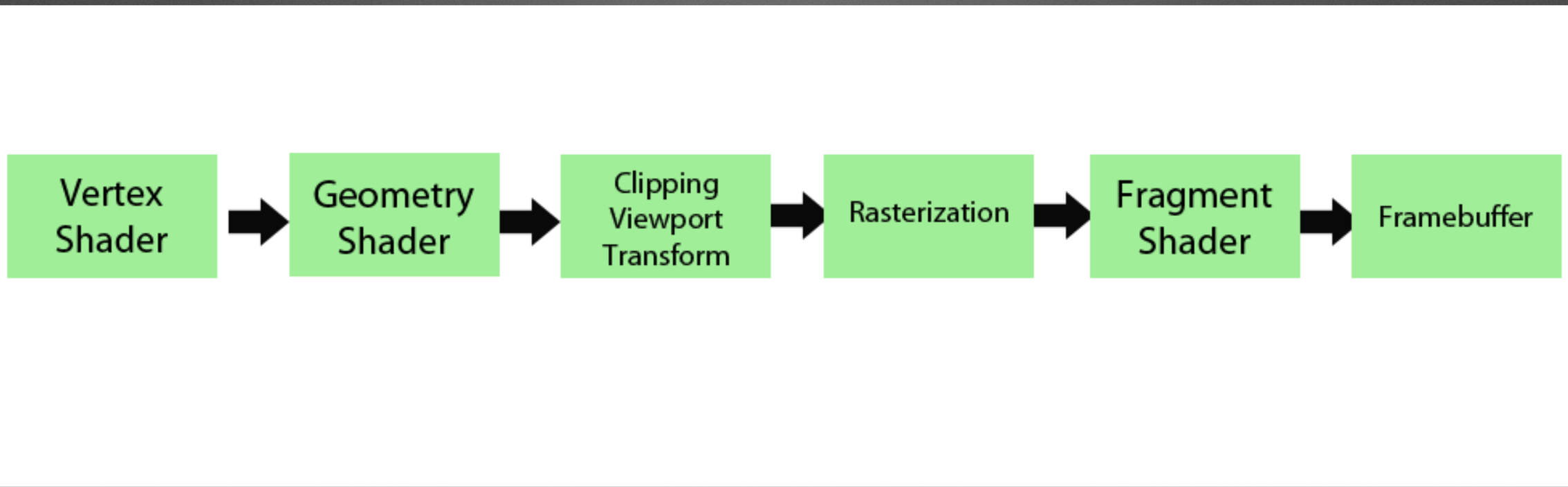
Programmable Vertex Shader

Programmable Fragment Shader

Geometry Shader

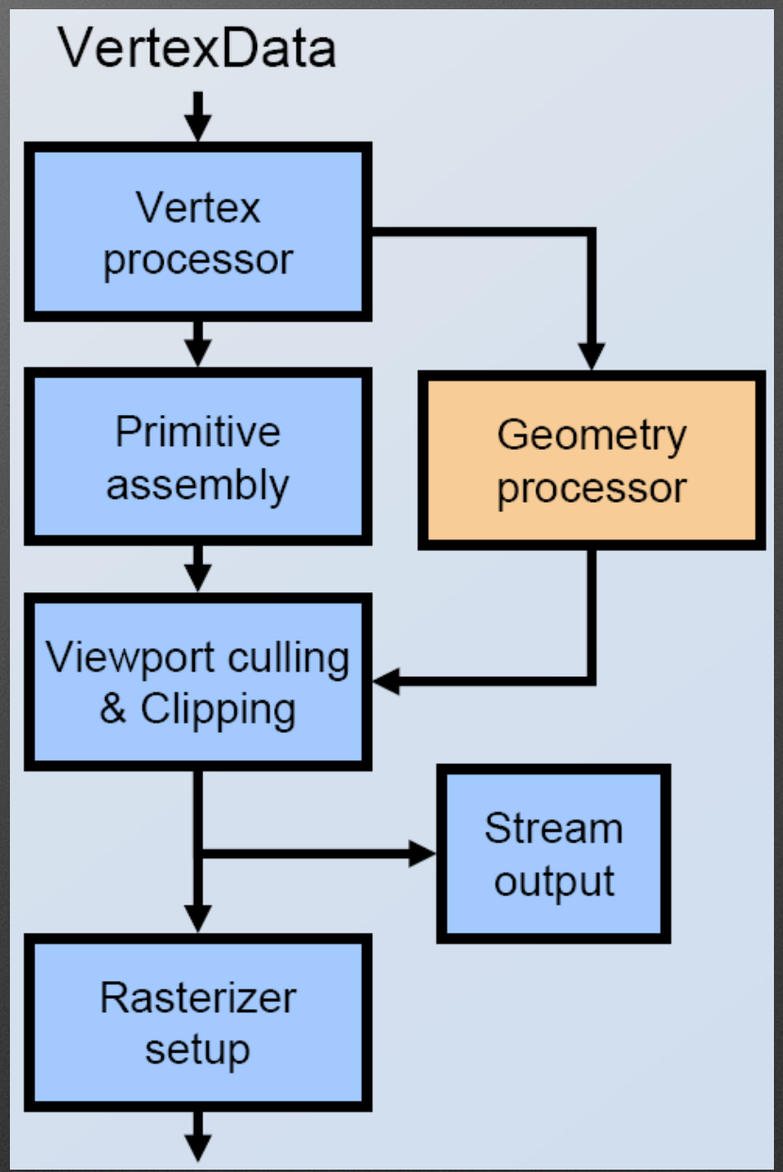
What is Geometry Shader?

Pipeline


Vertex Shader
Geometry Shader
Clipping Viewport Transform
Rasterization
Fragment Shader
Framebuffer

Geometry Shader

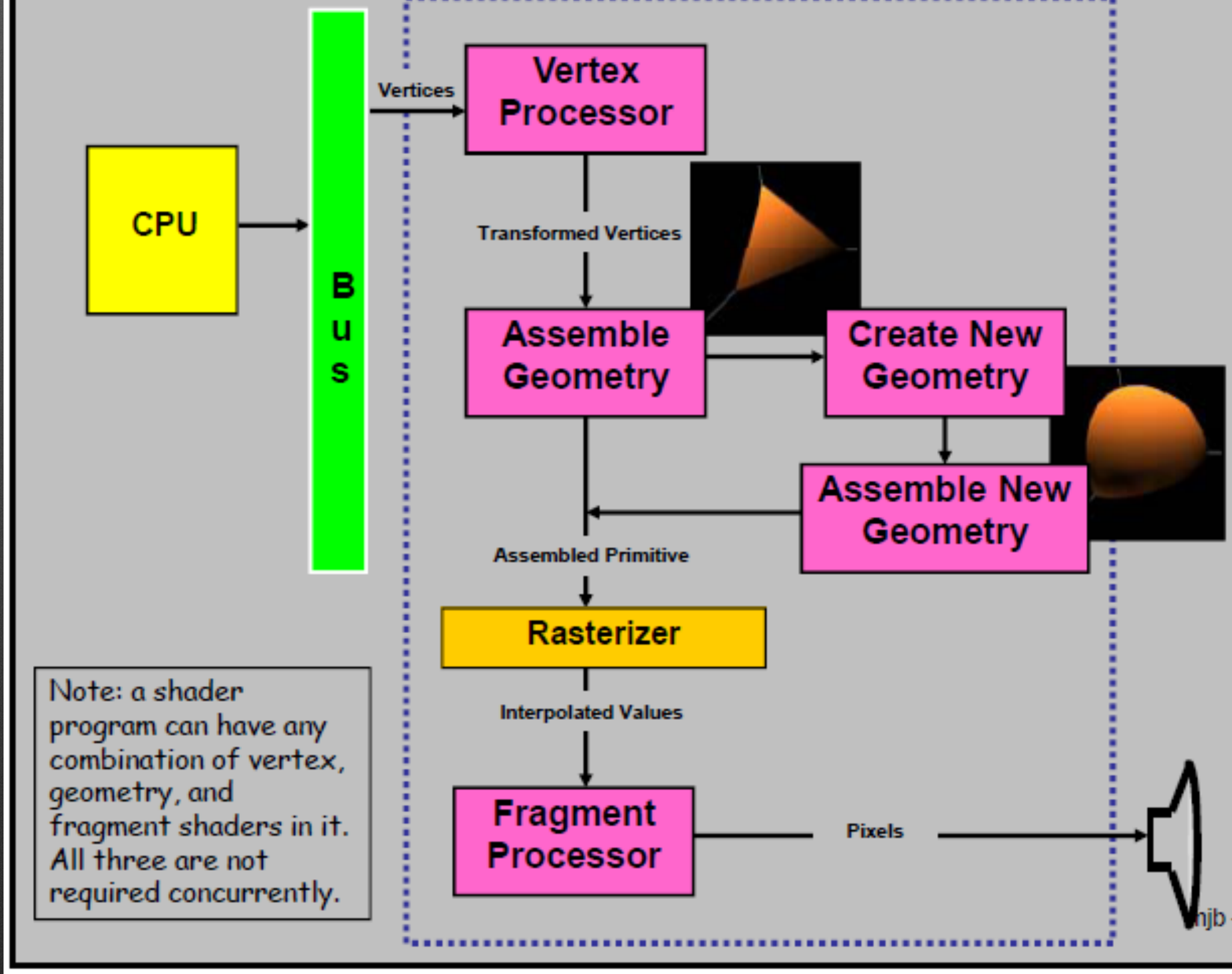
- Add/remove primitives
- Add/remove vertices
- Edit vertex position



What is a Geometry Shader?

- One primitive at a time
- Manipulate, Remove, or Create Primitives
- Discards original primitive
- Result is new set of transformed vertices
  - Clip coordinates
- Does not replace vertex shader
- Input and output primitive types don’t have to match
- August 3, 2009: OpenGL 3.2 as core functionality

The Geometry Shader: Where Does it Fit in the Pipeline?



Note: a shader program can have any combination of vertex, geometry, and fragment shaders in it. All three are not required concurrently.

Setup

- GLchar GeometryShaderSource; //load shader source
- GLint objectGS;
- objectGS = glCreateShader(GL_GEOMETRY_SHADER);
- glShaderSource(objectGS, 1, &GeometryShaderSource, NULL);
- glCompileShader(objectGS);
- myShaderProgam = glCreateProgram()
- glAttachShader(myShaderProgram, objectGS);
- glAttachShader(myShaderProgram, objectVS);
- glAttachShader(myShaderProgram, objectFS);
- glUseProgram(myShaderProgram);

Simple Example

Vertex Shader

```
uniform vec4 LightPosition;

const float SpecularContribution = 0.3;
const float DiffuseContribution  = 1.0 - SpecularContribution;

out float LightIntensity;

void main(void)
{

    vec3 ecPosition = vec3 (gl_ModelViewMatrix * gl_Vertex);
    vec3 tnorm      = normalize(gl_NormalMatrix * gl_Normal);
    vec3 lightVec   = normalize(LightPosition.xyz - ecPosition);
    vec3 reflectVec = reflect(-lightVec, tnorm);
    vec3 viewVec    = normalize(-ecPosition);
    float diffuse   = max(dot(lightVec, tnorm), 0.0);
    float spec      = 0.0;

    if (diffuse > 0.0)
    {
        spec = max(dot(reflectVec, viewVec), 0.0);
        spec = pow(spec, 16.0);
    }
    LightIntensity  = DiffuseContribution * diffuse +
                      SpecularContribution * spec;

    gl_Position = ftransform();
}
```

Vertex Shader

```
uniform vec4 LightPosition;

const float SpecularContribution = 0.3;
const float DiffuseContribution  = 1.0 - SpecularContribution;

out float LightIntensity;

void main(void)
{

    vec3 ecPosition = vec3 (gl_ModelViewMatrix * gl_Vertex);
    vec3 tnorm      = normalize(gl_NormalMatrix * gl_Normal);
    vec3 lightVec   = normalize(LightPosition.xyz - ecPosition);
    vec3 reflectVec = reflect(-lightVec, tnorm);
    vec3 viewVec    = normalize(-ecPosition);
    float diffuse   = max(dot(lightVec, tnorm), 0.0);
    float spec      = 0.0;

    if (diffuse > 0.0)
    {
        spec = max(dot(reflectVec, viewVec), 0.0);
        spec = pow(spec, 16.0);
    }
    LightIntensity  = DiffuseContribution * diffuse +
                      SpecularContribution * spec;

    gl_Position = ftransform();
}
```

Vertex Shader

```
uniform vec4 LightPosition;

const float SpecularContribution = 0.3;
const float DiffuseContribution  = 1.0 - SpecularContribution;

out float LightIntensity;

void main(void)
{

    vec3 ecPosition = vec3 (gl_ModelViewMatrix * gl_Vertex);
    vec3 tnorm      = normalize(gl_NormalMatrix * gl_Normal);
    vec3 lightVec   = normalize(LightPosition.xyz - ecPosition);
    vec3 reflectVec = reflect(-lightVec, tnorm);
    vec3 viewVec    = normalize(-ecPosition);
    float diffuse   = max(dot(lightVec, tnorm), 0.0);
    float spec      = 0.0;

    if (diffuse > 0.0)
    {
        spec = max(dot(reflectVec, viewVec), 0.0);
        spec = pow(spec, 16.0);
    }
    LightIntensity  = DiffuseContribution * diffuse +
                      SpecularContribution * spec;

    gl_Position = ftransform();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Geometry Shader

```
layout (triangles) in;
layout (triangle_strip, max_vertices = 3) out;


uniform float ExplodeFactor;

in  float LightIntensity[3];

smooth out float LightIntensityF;
void main(void)
{
    vec3 face_normal = normalize (
          cross (gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz,
               gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz));

    for (int i = 0; i < gl_in.length(); i++)
    {
        gl_Position = gl_in[i].gl_Position + vec4 (ExplodeFactor * face_normal, 0.f);
        LightIntensityF = LightIntensity[i];
        EmitVertex();
    }
    EndPrimitive();
}
```

Fragment Shader

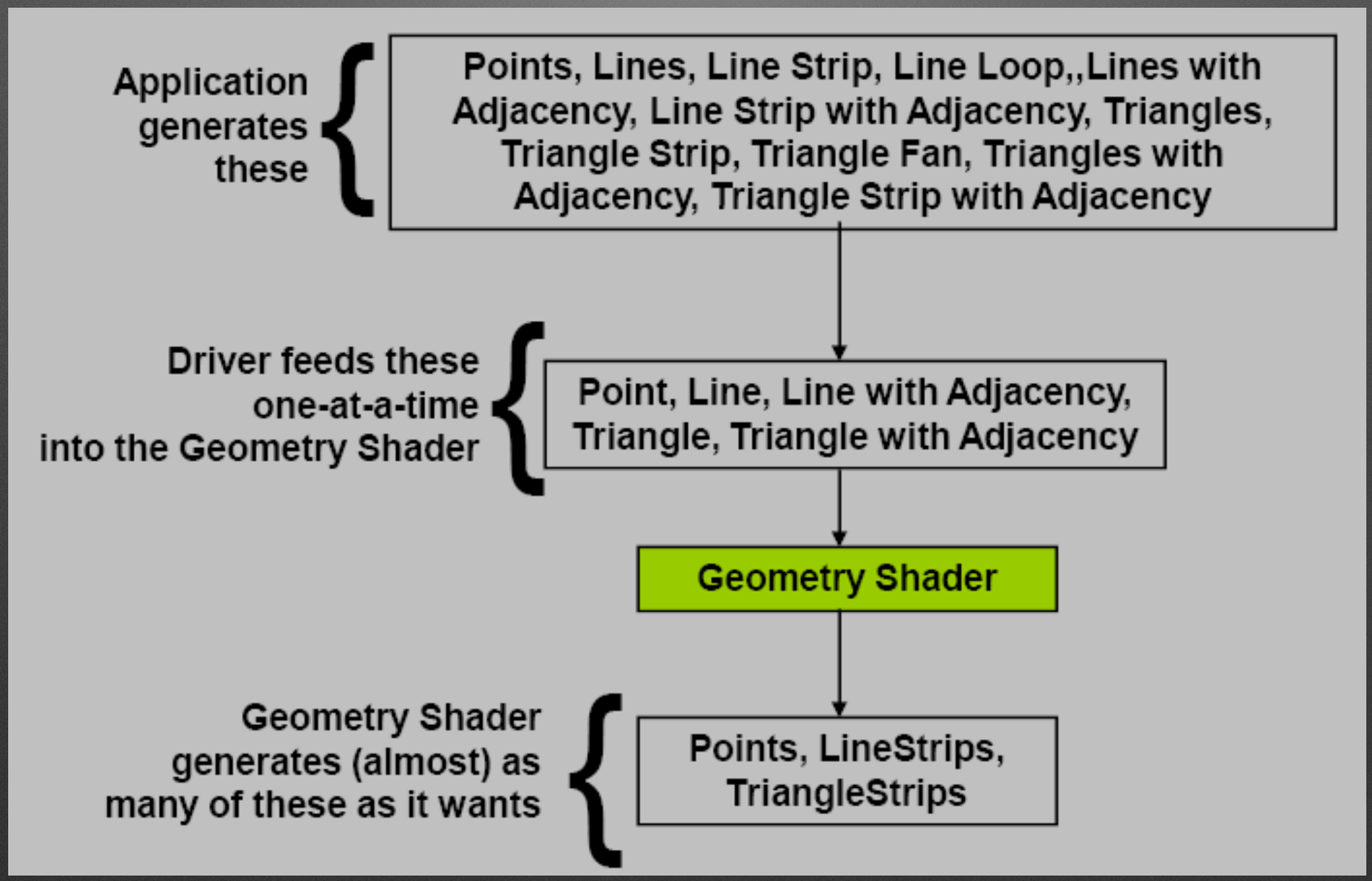
```
smooth in float LightIntensityF;

void main(void)
{
    vec3  color;

    color = vec3(1.0, 0.0, 0.0);
    color *= LightIntensityF;

    gl_FragColor = vec4 (color, 1.0);

}
```

Interpolation Qualifier

- Where to use?
  - Vertex shader outputs
  - Tessellation control shader inputs (to match with outputs from the VS)
  - Tessellation evaluation shader outputs
  - Geometry shader inputs (to match with outputs from the TES/VS) and outputs
  - Fragment shader inputs
- Types
  - flat: The value will not be interpolated. The value given to the fragment shader is the value from the Provoking Vertex for that primitive.
  - noperspective: The value will be linearly interpolated in window-space. This is usually not what you want, but it can have its uses.
  - smooth: The value will be interpolated in a perspective-correct fashion. This is the default if no qualifier is present.

Primitive In and Outs

Input/Output



Input Primitive Type

- Primitive type must match primitive mode

Input Types:

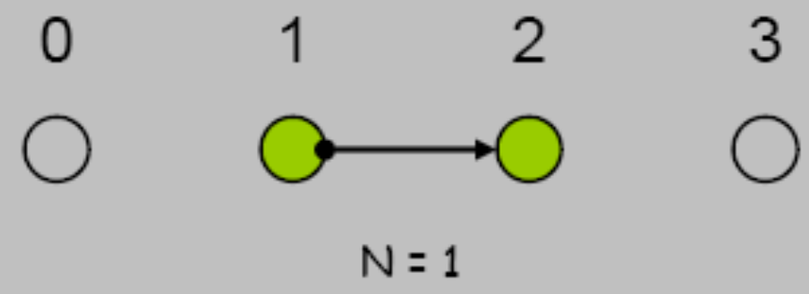
points
lines
lines_adjacency
triangles
triangles_adjacency

Draw Modes:

points
lines
lines_strip
line_loop
lines_adjacency
lines_strip_adjacency
triangles
triangle_strip
triangle_fan
triangles_adjacency
Triangle_strip_adjacency

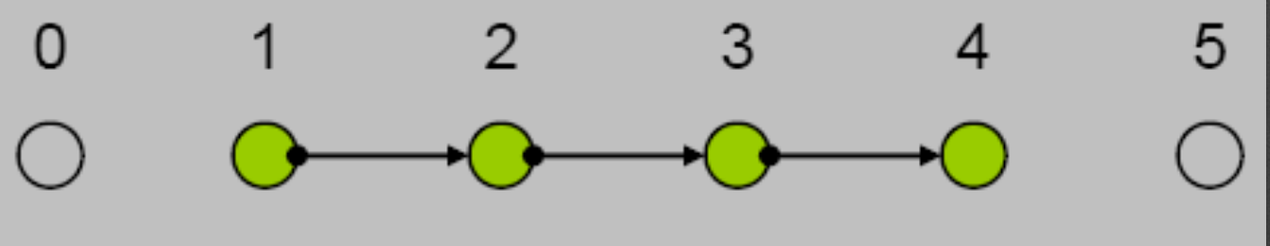
New (input) Adjacency Primitives

Lines with Adjacency



4N vertices are given.
(where N is the number of line segments to draw).
A line segment is drawn between #1 and #2.
Vertices #0 and #3 are there to provide adjacency information.

Line Strip with Adjacency

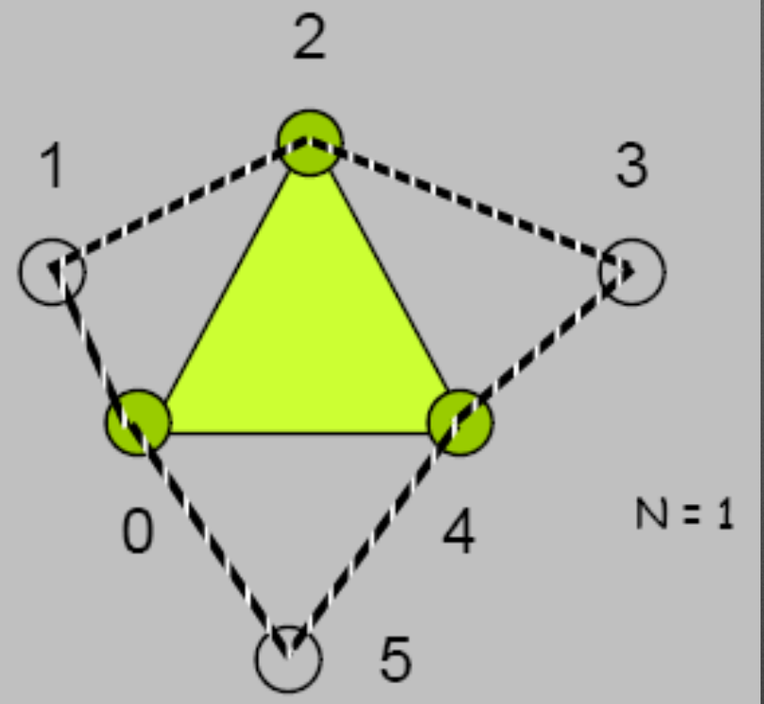


N+3 vertices are given
(where N is the number of line segments to draw).
A line segment is drawn between #1 and #2, #2 and #3, …, #N and #N+1.
Vertices #0 and #N+2 are there to provide adjacency information.

New (input) Adjacency Primitives

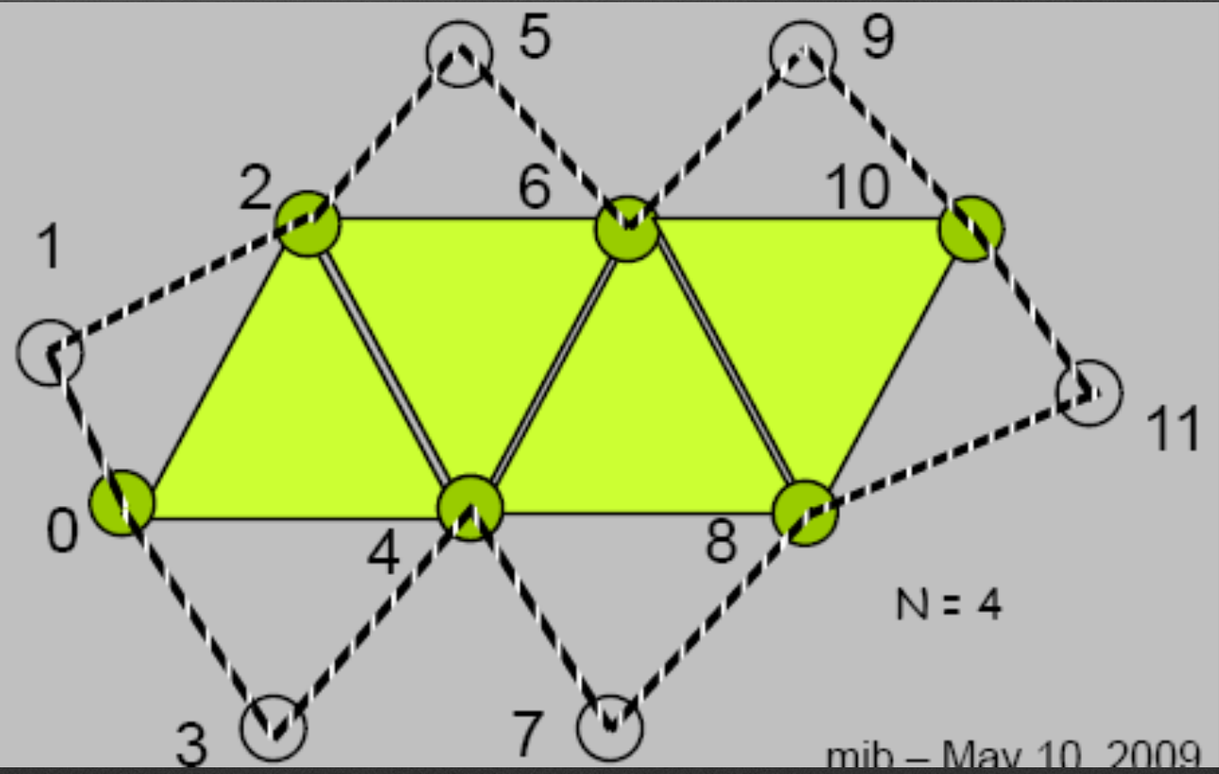
Triangles with Adjacency

6N vertices are given
(where N is the number of triangles to draw).
Points 0, 2, and 4 define the triangle.
Points 1, 3, and 5 tell where adjacent triangles are.



Triangle Strip with Adjacency

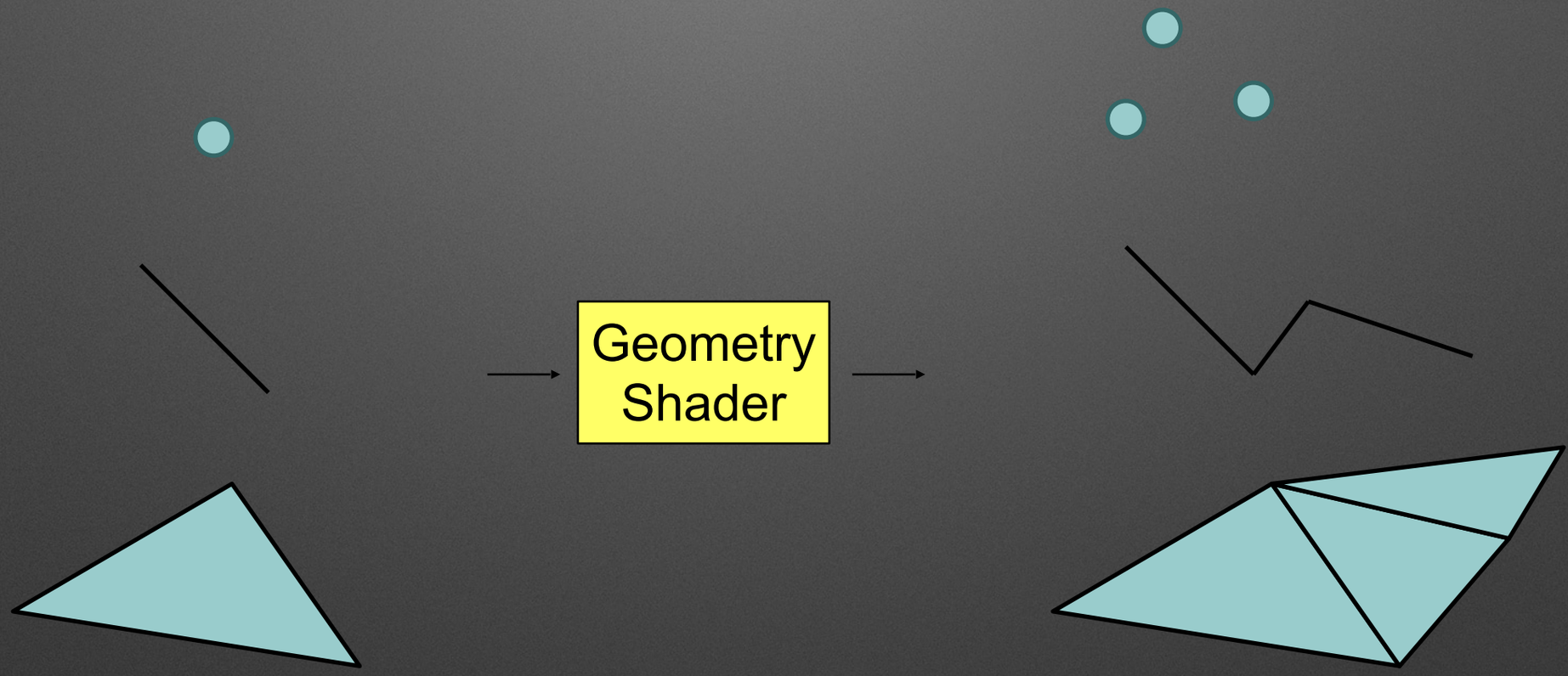
4+2N vertices are given
(where N is the number of triangles to draw).
Points 0, 2, 4, 6, 8, 10, …define the triangles.
Points 1, 3, 5, 7, 9, 11, … tell where adjacent triangles are.



Built-In Input Variables

- gl_PrimitiveIDIn
  - number of primitives processed so far
  - set in primitive assembly stage
- gl_in[]
  - gl_Position – position from vertex shader, transformed?
  - gl_PointSize – size in pixels
  - gl_ClipDistance[] – user defined clip distance

Primitive Types



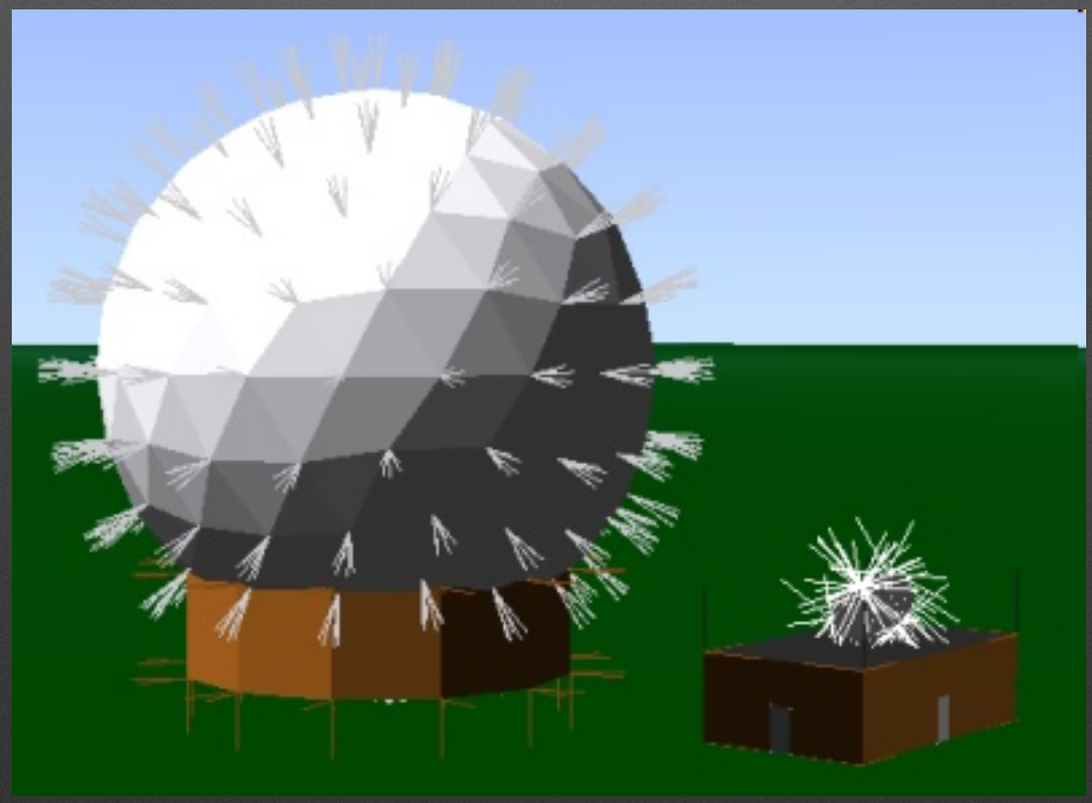
Output primitives can be disconnected

Primitive Types

- Input Primitives
  - GL_POINTS
  - GL_LINES
  - GL_TRIANGLES
  - Adjacency
- Output Primitives
  - GL_POINTS
  - GL_LINE_STRIP
  - GL_TRIANGLE_STRIP

Primitive Types

- Input primitive type doesn't have to equal output primitive type



Geometry Shader Example

Geometry Language

Inputs

```
in gl_PerVertex {
    vec4   gl_Position;
    float  gl_PointSize;
    float  gl_ClipDistance[];
    float  gl_CullDistance[];
} gl_in[];
in int gl_PrimitiveIDIn;
in int gl_InvocationID;
```

Outputs

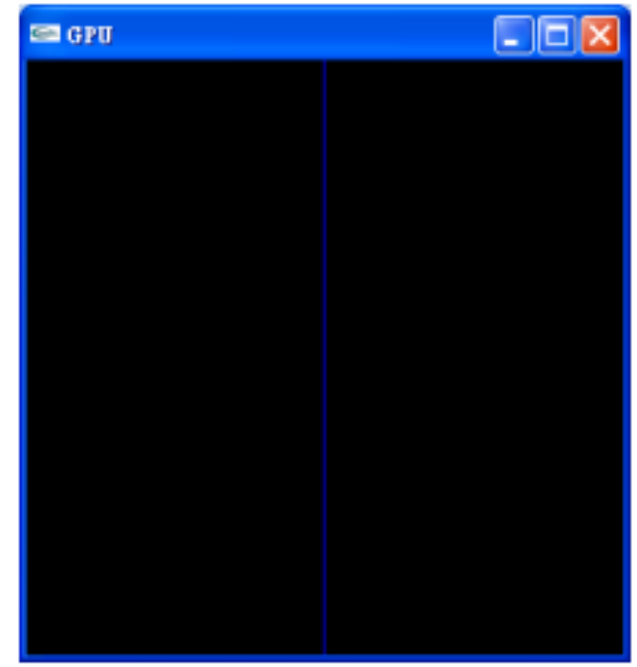
```
out gl_PerVertex {
   vec4  gl_Position;
   float  gl_PointSize;
   float  gl_ClipDistance[];
   float  gl_CullDistance[];
};
out  int  gl_PrimitiveID;
out  int  gl_Layer;
out  int  gl_ViewportIndex;
```

Create Lines from Points

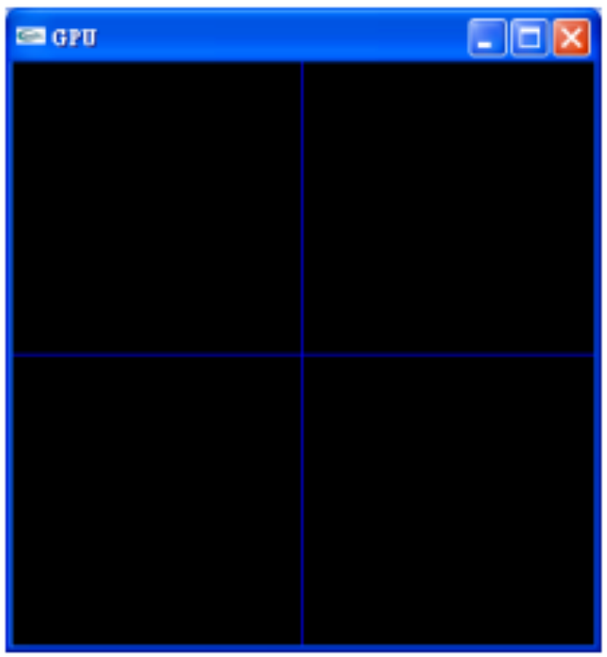
```
#version 330 core
layout (points) in;
layout (line_strip, max_vertices = 2) out;

void main() {
    gl_Position = gl_in[0].gl_Position + vec4(-0.1, 0.0, 0.0, 0.0);
    EmitVertex();

    gl_Position = gl_in[0].gl_Position + vec4( 0.1, 0.0, 0.0, 0.0);
    EmitVertex();

    EndPrimitive();
}
```

Line + Line

```
#version 330 core
layout (lines) in;
layout (line_strip, max_vertices = 4) out;

void main() {
  int i;
  for (i = 0; i < 2; i++) {
    gl_Position = gl_in[i].gl_Position;
    EmitVertex();
  }
  EmitPrimitive();

  for (i = 0; i < 2; i++) {
    gl_Position = gl_in[i].gl_Position;
    gl_Position.xy = gl_Position.yx;
    EmitVertex();
  }
  EmitPrimitive();

}
```



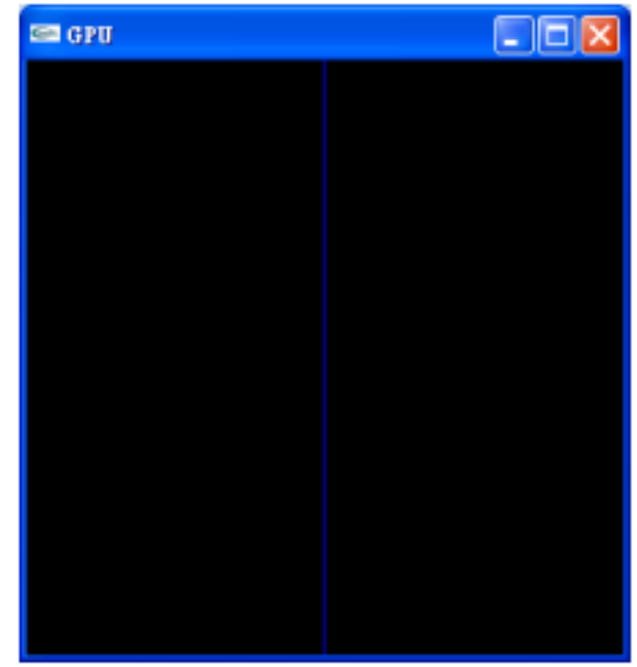
Original input primitive



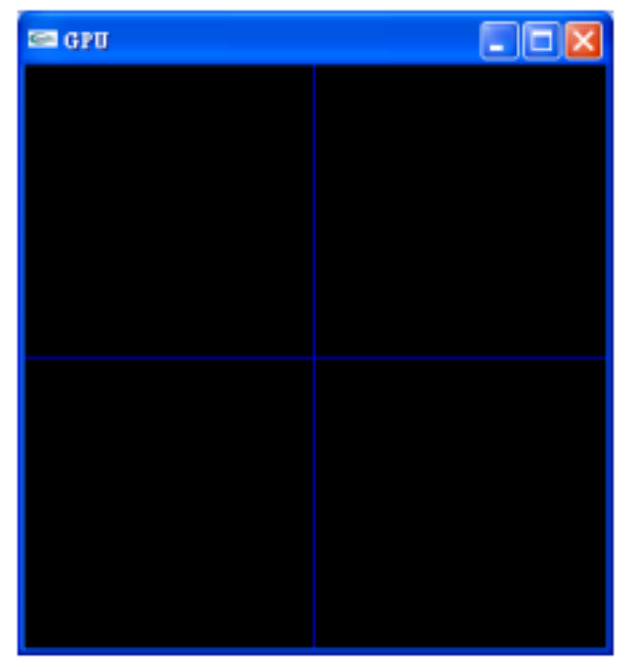
Output primitive

Line + Line With Color

```
#version 330 core
layout (lines) in;
layout (line_strip, max_vertices = 4) out;

in vec3 color[3];

out vec3 colorOut;

void main() {
  int i;

  for (i = 0; i < 2; i++) {
    gl_Position = gl_in[i].gl_Position;
    colorOut = color[i];
    EmitVertex();
  }
  EmitPrimitive();

  for (i = 0; i < 2; i++) {
    gl_Position = gl_in[i].gl_Position;
    gl_Position.xy = gl_Position.yx;
    colorOut = color[i];
    EmitVertex();
  }
  EmitPrimitive();

}
```



Original input primitive



Output primitive

Bezier Curve (Simple Example)

- Vertex Shader

```
#version 330 core

in vec3 position;
in vec3 color;

out vec3 v2gColor;

void main() {
  gl_Position = position;
  v2gColor = color;
}
```

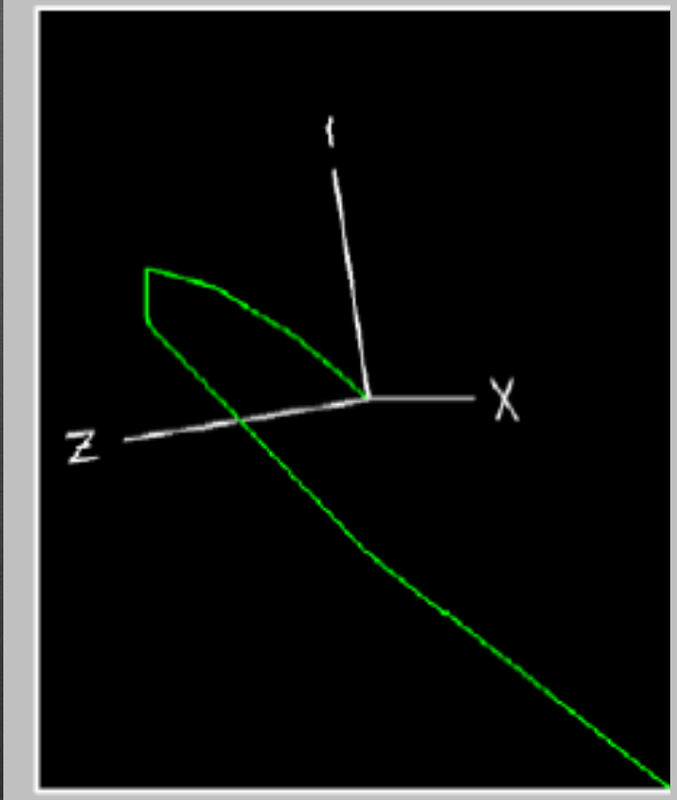
- Fragment Shader

```
#version 330 core

in vec3 g2fColor;
out vec3 colorOut;

void main() {
  colorOut = g2fColor;
}
```

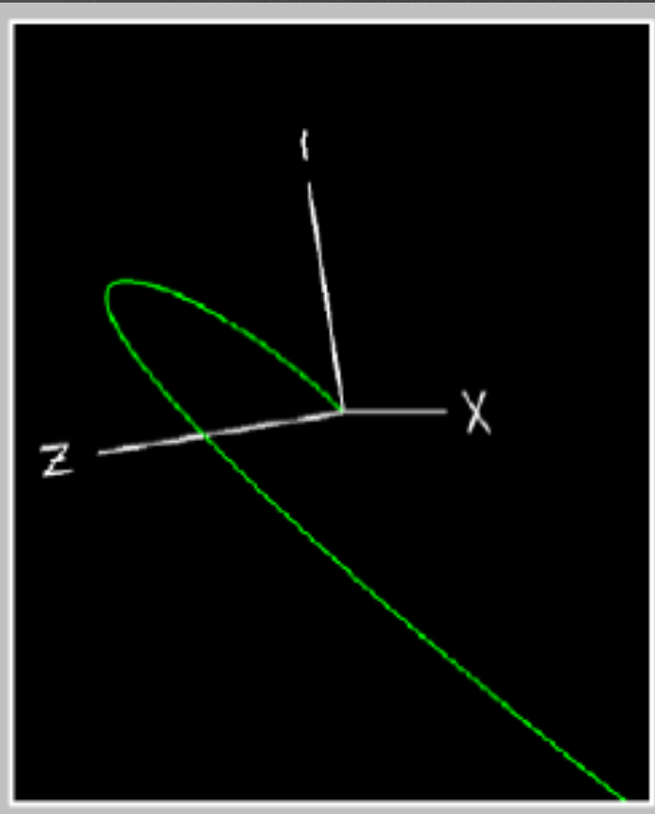
Bezier Curve (Simple Example)

- Geometry Shader

```
#version 330 core

layout (lines_adjacency) in; // How to draw LINES_ADJACENCY ? Look glDrawArrays or glDrawElements manual
layout (line_strip, max_vertices = 128) out;

uniform int tess;

out vec3 g2fColor;

void main() {
   float du = 1.0/float(tess);
   float u = 0;
   for (int i = 0; i<=tess; i++)
   {
      float term1 = 1.0 - u;
      float term2 = term1*term1;
      float term3 = term1*term2;
      float u2 = u*u;
      float u3 = u*u2;
      vec4 p = term3 * gl_in[0].gl_Position
             + 3.0 * u * term2 * gl_in[1].gl_Position
             + 3.0 * u2 * term * gl_in[2].gl_Position
             + u3 * gl_in[3].gl_Position;
      gl_Position = p;
      EmitVertex();
      u += du;
   }
   EmitPrimitive();
}
```



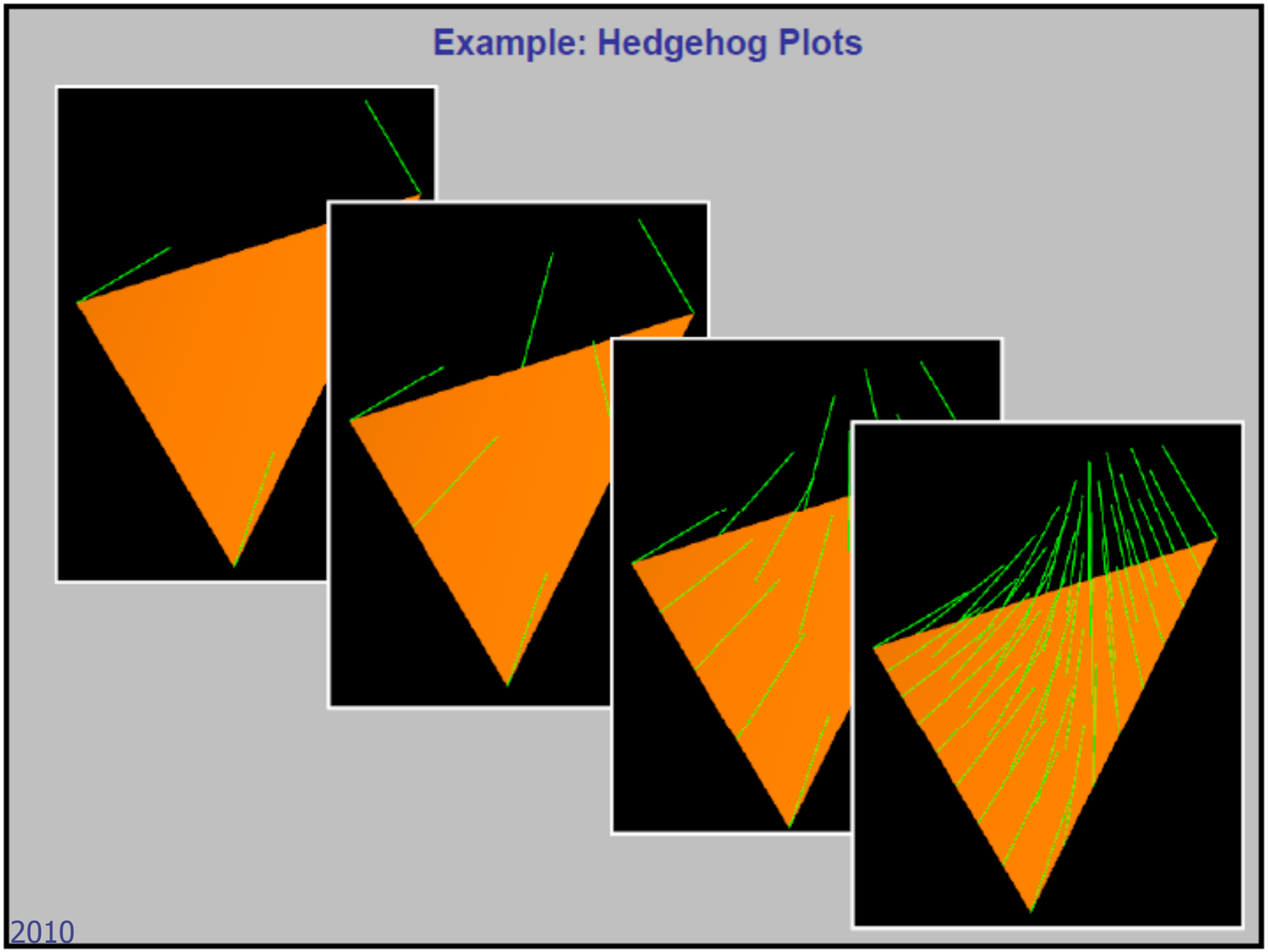
FpNum = 5



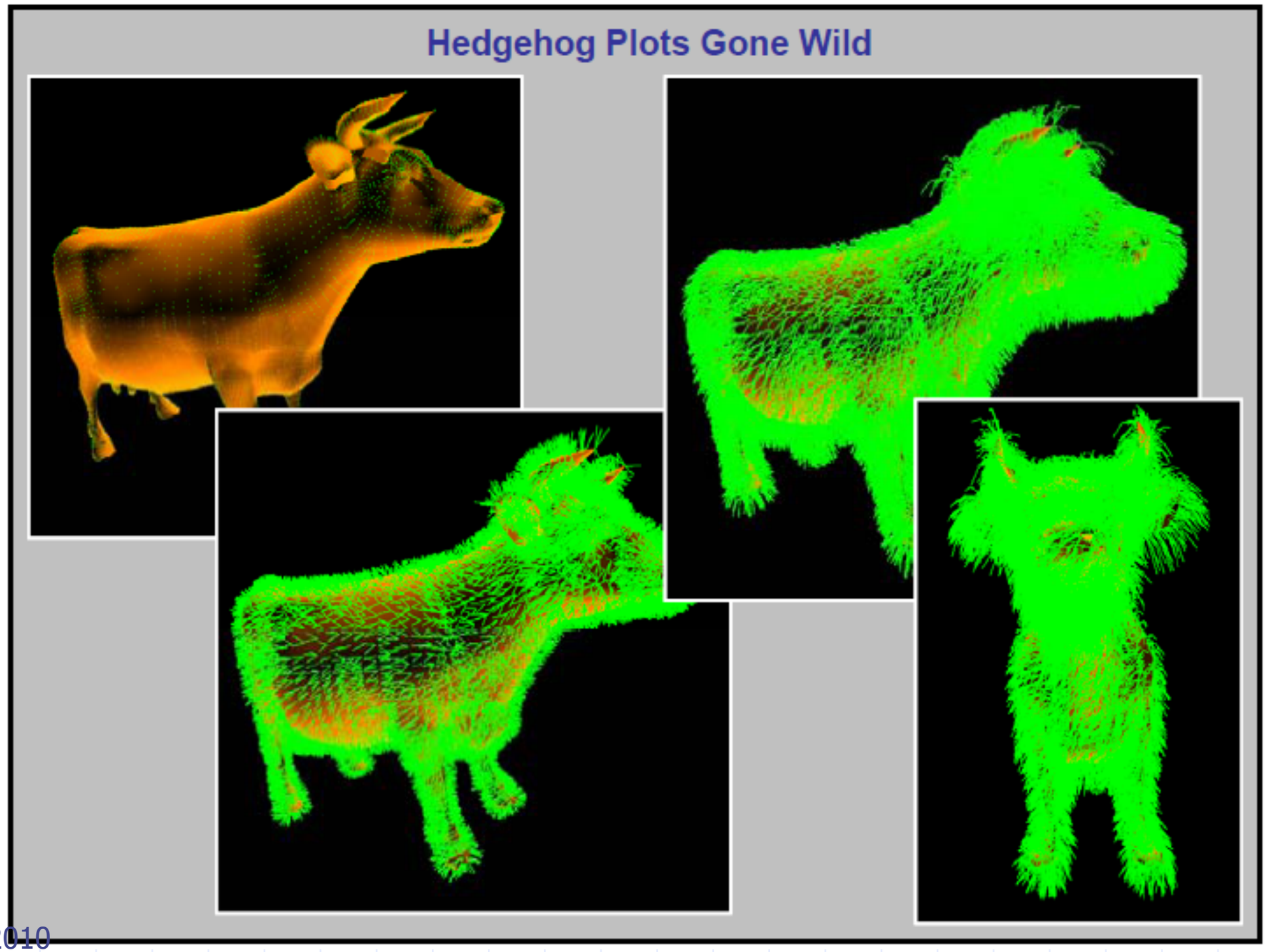
FpNum = 25

More Applications

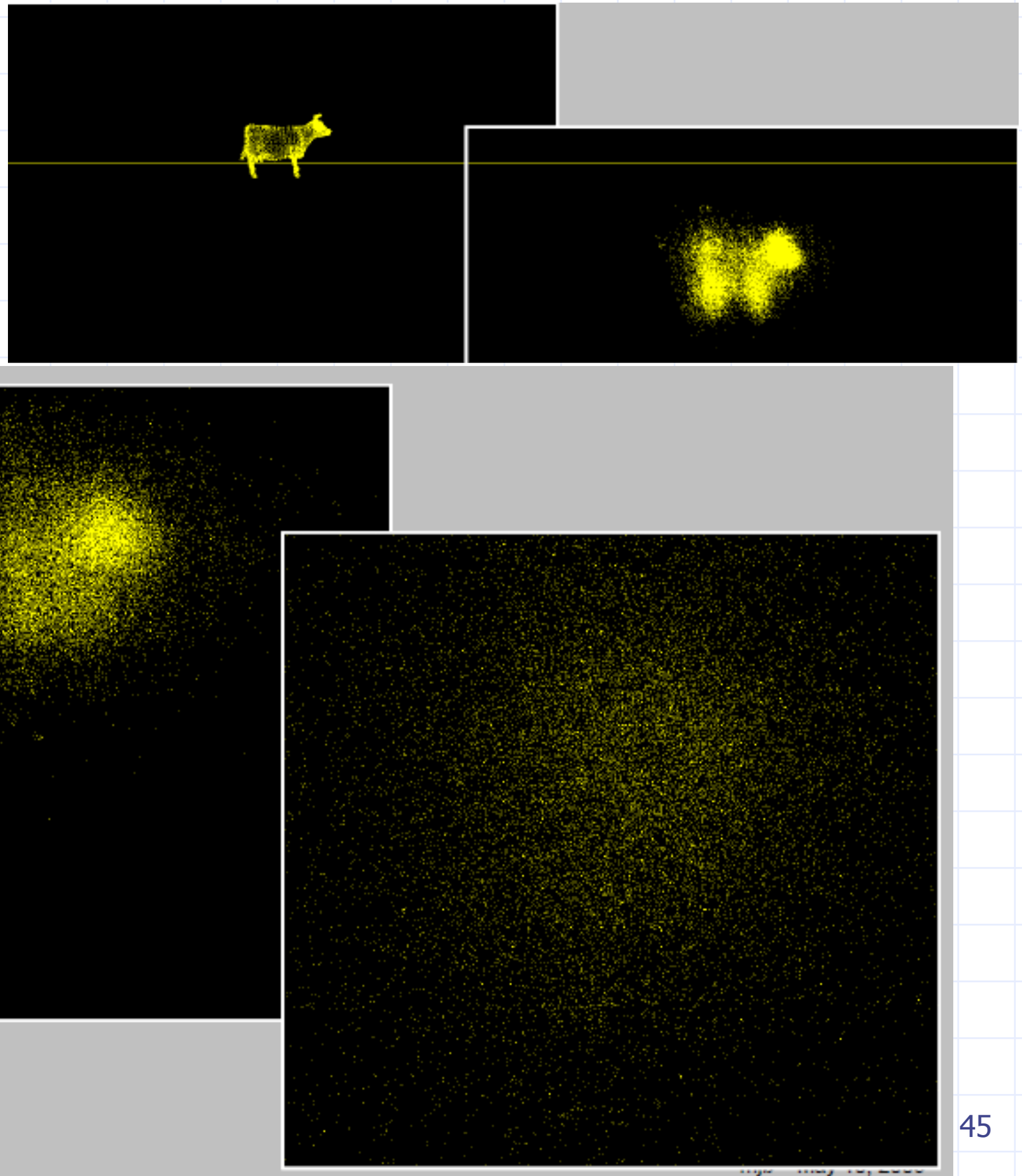
Hedgehog Plots



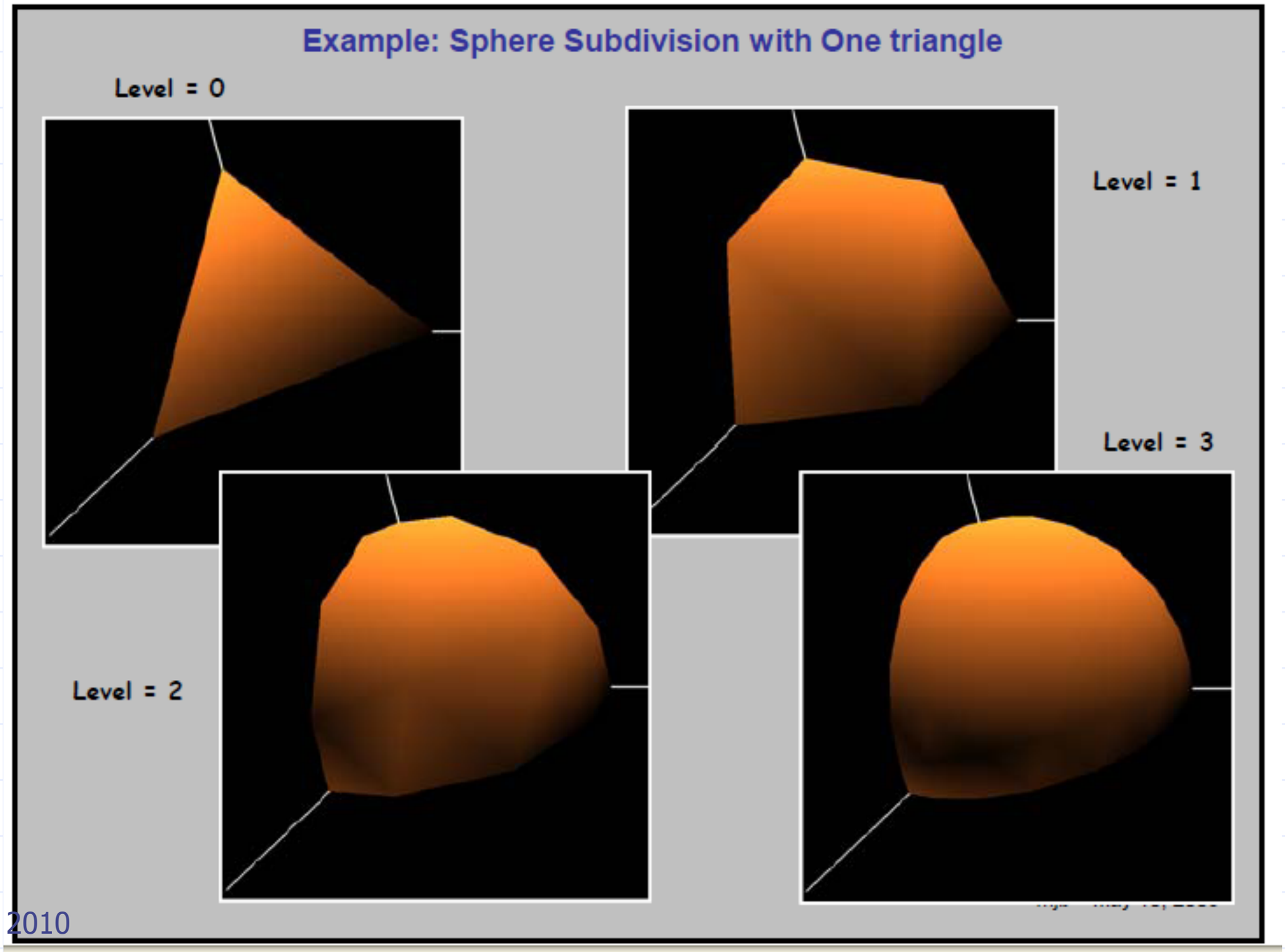
Hedgehog Plots



Explosion



Subdivision Surface



Homework

- Create a Bezier curve drawing code using Geometry Shader
  - Input: 4 points, tessellation level
  - Output: Bezier curve
- 제출물:
  - Tessellation level 10 과 100 에 대하여 각각 그린 Screen shot
  - 소스파일 (C/C++ 및 Shader file 모두 제출)
  - 주의: Geometry Shader 를 사용하지 않으면 0점 처리

Create a Geometry Shader

- glCreateShader(GL_GEOMETRY_SHADER);
- glShaderSource
- glCompileShader
- glAttachShader
- glLinkProgram

End